

3.01. Le Père Noël

3.01 : le père Noël — style manga : grands yeux, bonnet, barbe. Dessiné une fois en 16×16 , montré à sa taille puis trois fois plus grand avec `sprite16(..., 3)`.

1. Ce qu'il faut comprendre

Le Père Noël : style manga : grands yeux, bonnet, barbe. Une image de 16×16 , dessinée **une seule fois**, dans `perso_PERE_NOEL.cpp`.

Cette série montre des images, une par leçon, et les jeux qui s'en servent : Puissance 4, Dames, Échecs, et un jeu de cartes (dans le menu des exemples). Le programme est le même d'une leçon à l'autre : seuls changent le dessin et le nom.

À gauche, l'image à sa taille : `sprite16(0, 24, 56, PERE_NOEL)`, les lutins 0 à 3.

À droite, la même, trois fois plus grande : `sprite16(4, 88, 36, PERE_NOEL, 3)`. Le **3**, en dernier argument, est la **taille** : le compilateur redessine l'image en 48×48 avant que le jeu ne tourne, en gardant les vrais coins pointus et en arrondissant les marches. C'est la logique du 2.40, écrite directement dans `sprite16()`.

Combien de lutins ? 48×48 , c'est 6×6 carrés de 8 : 36 lutins au plus (les carrés vides ne comptent pas), à partir du lutin 4. Avec les 4 de la petite image : 40, tout ce que la console a. Plus grand encore, la forme passe d'elle-même dans le fond de l'écran : il n'y a pas de limite de taille.

Les petits détails (un œil d'un pixel) restent petits : l'agrandissement suit les formes, il n'invente rien. Plus le dessin de 16×16 est net, plus le grand est beau.

Essaie : change le `3` en `2`, puis en `5` (elle passe alors dans le fond, et le premier argument ne sert plus). Puis modifie un pixel dans le dessin : les deux tailles changent ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite16>` → [35.2. La fonction sprite16\(\) — un personnage de \$16 \times 16\$](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1789 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	876 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre

```

RangerLutins      11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles       67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage – vide les deux cartes de l'écran
Tuiles             656 o ← CopierTuiles – 6 lettres sur 43, et 35 tuiles dessinées
DonneesTransfert   10 o ← InstallerTransfert – la petite routine qui copie les lutins
"PERE NOEL"        9 o ← texte() – un texte gravé

```

Lettres gravées : ELNOPR

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```

sprite()      x 37   855 o sur place, dans main()
texte()       x 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()       x 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16()    x 2     0 o sur place, dans main()
spriteTaille() x 1     0 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- 3.01 Le Père Noël ----
// Dessiné UNE fois, en 16 × 16. À gauche à sa taille, à droite trois fois plus grand.

#include <texte>           // texte() : écrire un mot
#include <sprite16>        // sprite16() : poser un dessin de 16 × 16 – et l'agrandir
#include <Perso>           // Perso : un dessin de 16 × 16
#include "perso_PERE_NOEL.cpp" // le dessin, dans son fichier

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

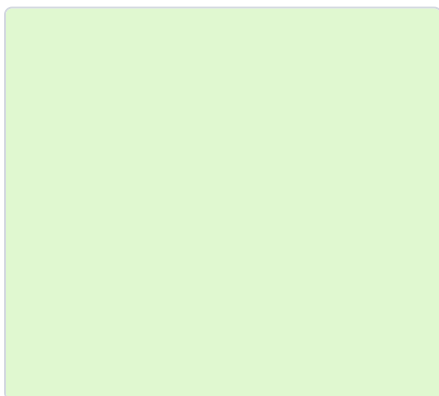
int main() {
    sprite16(0, 24, 56, PERE_NOEL);    // taille 1 : les lutins 0 à 3
    sprite16(4, 88, 36, PERE_NOEL, 3); // taille 3 : 48 × 48, à partir du lutin 4
    texte(5, 12, "PERE NOEL");

    while (true) {
        image();
    }
}

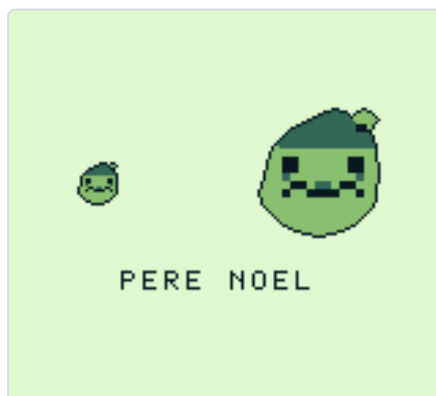
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le Père Noël deux fois : petit à gauche, trois fois plus grand à droite, et PERE NOEL dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ taille 1 : l'image en (24, 56)
- ✓ taille 3 : des lutins dans le carré de 48 à droite (lutin 4 en 96, 36)
- ✓ le nom est écrit : PERE NOEL

Ensuite : **3.02. Le bonhomme de neige** — série 3 : Les images. · `gameboy3` — écrire une cartouche Game Boy en C++